SIM Swap Fraud Detection in Kenya's M-Pesa Ecosystem

Project Documentation Report: a supervised machine learning build from problem framing to a deployed two-layer fraud scoring app

Author: Urbanus Kathitu (Kat.Codes)

Build period: June 2026 to October 2026

Report date: October 2026

Live app: Streamlit Community Cloud

Source code: github.com/KathituCodes/SIM-Swap-Fraud-Detection-in-M-Pesa

Dataset: Kaggle: M-Pesa Transactions Fraud (synthetic)

1. Executive Summary

Kenya's mobile money ecosystem is under sustained attack from SIM swap fraud. A criminal convinces a mobile operator to move a victim's phone number onto a SIM card they control, then uses the one-time passwords sent to that number to reset PINs, empty the M-Pesa wallet and draw down every credit line attached to it. Losses reported by Kenyan banks through mobile channels more than quadrupled in 2024, and INTERPOL's 2026 assessment reports a 327% rise in SIM swap fraud in Kenya.

This project asked a practical question: can transaction data alone flag a SIM swap style drain at the moment it happens, and can the decision be explained to a regulator? It produced:

- **Four engineered behavioural features** that turn raw balances into SIM swap signals, since the dataset contains no native SIM swap fields.
- **Three detection models** (Logistic Regression baseline, XGBoost primary, LightGBM comparison) trained on 120,000 synthetic M-Pesa transactions with a 33:1 class imbalance.
- **A Weight of Evidence (WoE) logistic regression scorecard** that explains every flag feature by feature, the standard approach in regulated credit scoring, plus PSI drift monitoring.
- **A deployed two-layer Streamlit app** on Streamlit Community Cloud: XGBoost for real-time blocking and the scorecard for audit trails.

Model	Precision	Recall	F1	AUC-ROC	False positives
XGBoost (primary)	0.99	0.66	0.79	0.82	4
LightGBM	0.96	0.65	0.78	0.83	19
WoE Scorecard (regulatory)	0.40	0.67	0.50	0.83	712
Logistic Regression (baseline)	0.52	0.66	0.58	0.83	426

Results on the 24,000-transaction held-out test set (702 fraud cases).

> **Headline finding.**
>
> XGBoost flags fraud with 0.99 precision (4 false positives in 24,000 transactions), but recall stalls at 0.66 against a 0.80 target. A sanity check showed why: a single rule, "flag when the sender's balance after the transaction is below KES 100", reaches 0.98 precision and 0.63 recall on its own and agrees with XGBoost on 99.87% of test rows. Most of the signal in this synthetic dataset comes from one balance wipeout pattern. The remaining third of fraud looks like ordinary traffic and needs signals the dataset does not contain, such as SIM swap timestamps and device changes.

2. Background: Why SIM Swap Fraud in Kenya

The project started from a simple observation: Kenya runs one of the world's most successful mobile money systems, and the same phone number that makes M-Pesa convenient is also its single point of failure. Once a fraudster controls the number, they control the wallet, the PIN reset and every loan product linked to it. The evidence below, gathered from public reporting, shaped the problem statement.

2.1 Scale of the problem

- **Bank fraud losses quadrupled in 2024.** Central Bank of Kenya data reported in the press shows customers lost KSh 1.59 billion to fraud in 2024 (from KSh 412 million in 2023). Mobile banking losses reached KSh 810.68 million, a 344% increase, with SIM swap schemes named among the methods. (TechTrends Kenya; Business Daily)
- **SIM swap fraud up 327%.** INTERPOL's African Cyberthreat Assessment 2026 reports a 327% surge in SIM swap fraud in Kenya, more than 123,000 fraudulently issued SIMs and about USD 3.8 million drained from mobile wallets. (The Kenya Times; Business Daily)
- **Credit products are the bigger prize.** In 2023 the DCI's Banking Fraud Investigations Unit arrested eight suspects over a scheme that used fraudulently registered SIM cards to take more than KSh 500 million in Fuliza overdrafts. Over 123,000 new numbers opted into Fuliza and borrowed before being discarded. (Citizen Digital)
- **Kenya is a high-risk market.** Mobile payment security firm Evina found 51% of mobile transactions in Kenya flagged as suspect. (Connecting Africa) The government's NC4 committee reported in 2026 that mobile money featured in half of the computer fraud cases it reviewed. (Techweez)

2.2 Liability is shifting to operators and banks

In July 2026 the High Court in Machakos upheld a ruling that Safaricom and Diamond Trust Bank must jointly pay a SIM swap victim KSh 4.4 million, split 60/40. The court found Safaricom's weak verification of the SIM swap request was the proximate cause of the loss, and that the bank should have flagged rapid transfers to multiple unknown accounts instead of relying on a correct PIN. (HapaKenya) The message to the industry is that reactive security is no longer a defensible position.

2.3 How the attack works

Reporting by Rest of World and the INTERPOL assessment describes a consistent pattern:

1. The fraudster gathers the victim's ID details through social engineering, data leaks or fake customer care accounts.
2. They persuade operator staff, or a corrupt insider, to issue a replacement SIM for the victim's number.
3. The victim's phone loses signal. OTPs and alerts now go to the fraudster.
4. The fraudster resets the M-Pesa PIN, moves the balance out and draws down linked credit (Fuliza, M-Shwari, KCB M-Pesa, bank apps), often within an hour.

Existing defences such as customer-reported SIM blocks, manual freezes and post-hoc investigations all depend on someone noticing first. That is the gap this project targets: a proactive layer that scores each transaction as it happens.

> **Source note.** An earlier version of the notebook cited the KSh 810 million figure as coming from the "CBK 2024 Financial Sector Stability Report". The figure is confirmed in press coverage of CBK data, but the exact CBK publication was not independently verified for this report. Cite the CBK report directly once the primary document is located.

3. Objectives and Success Metrics

3.1 Objectives

1. Detect fraudulent M-Pesa transactions that follow a SIM swap behavioural pattern before the financial damage is complete.
2. Engineer proxy features that simulate SIM swap signals from available transaction data.
3. Train a supervised classifier that handles a realistic 33:1 class imbalance.
4. Build a regulator-friendly scorecard that explains each decision, alongside the primary model.
5. Deploy a fraud scoring system that scores transactions in real time.

3.2 Target users

- **Fintech operators and digital lenders** (Safaricom, NCBA, KCB and similar) who carry the non-performing loans from fraudulent Fuliza and M-Shwari drawdowns.
- **End users** who expect protection without friction on legitimate transactions.
- **Regulators** (CBK, Communications Authority) who need auditable, explainable fraud controls.

3.3 Success metrics: targets versus results

Metric	Target	Best achieved	Status	Why it matters
Recall	0.80	0.67 (WoE)	Not met	Missed fraud costs more than a false alarm
Precision	0.70	0.99 (XGBoost)	Met	Limits wrongly blocked customers
AUC-ROC	0.90	0.83 (LR)	Not met	Overall separation of fraud and legitimate
F1-Score	Maximise	0.79 (XGBoost)	Best of four	Balance of precision and recall

Accuracy was deliberately excluded. With 97.1% of transactions legitimate, a model that never flags fraud would still score 97% accuracy.

4. Data

4.1 Source and structure

The project uses the M-Pesa Transactions Fraud dataset on Kaggle, a synthetic dataset of 120,000 transactions and 13 columns. It was treated as if it were real data, with no assumptions targeting specific regions, devices or demographics.

Column	Type	Description
transaction_id	text	Unique ID; prefix encodes month. Dropped before modelling.
amount	number	Transfer amount in KES
sender_balance_before / after	number	Sender wallet balance before and after the transaction
receiver_balance_before / after	number	Receiver wallet balance before and after
transaction_type	category	peer, paybill or till
hour	integer	Hour of day, 0 to 23
month_2026	integer	Synthetic month artefact. Dropped.
day_of_week	category	Mon to Sun
device_type	category	smartphone or feature phone (USSD)
region	category	Nairobi, Mombasa, Kisumu, Nakuru, Eldoret
is_fraud	0 / 1	Target label (SIM swap fraud = 1)

4.2 Data quality checks

- Zero missing values across all columns and zero duplicate rows or transaction IDs.
- One zero-amount transaction, removed during preparation (119,999 rows remain).
- No negative amounts, no invalid hours, and no sender balances that increased after sending.
- 5,778 amount outliers (4.8%) by the IQR rule. Their fraud rate is 8.97% against 2.93% overall, so they were kept as genuine signal rather than removed as errors.
- An automated ydata-profiling report was generated and saved as mpesa_profile_report.html.

4.3 Class imbalance

116,490 legitimate transactions (97.1%) against 3,510 fraud cases (2.9%), a ratio of 33.2:1. This drove every later decision about splitting, weighting and metrics.

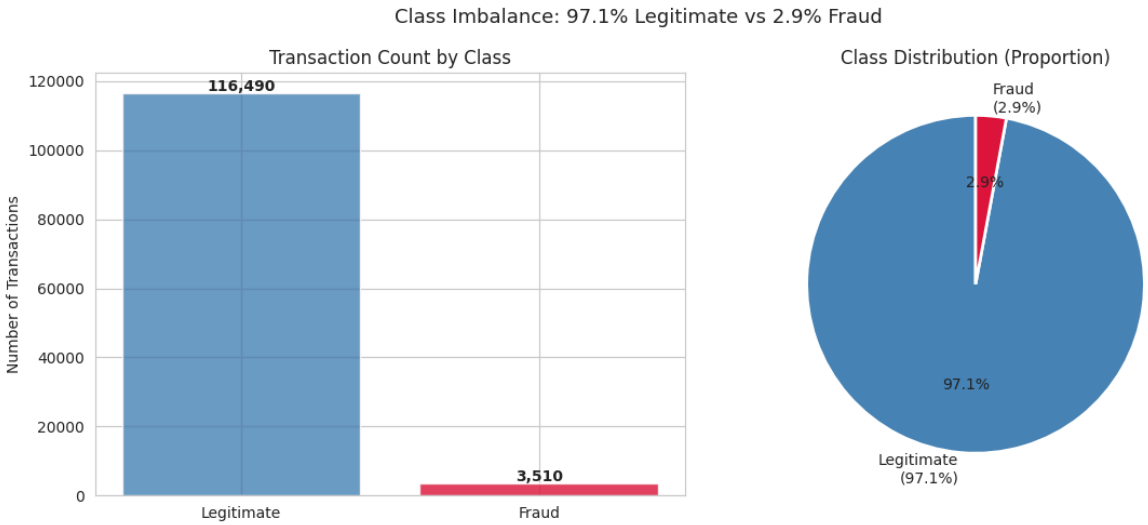


Figure 1. Class distribution: 97.1% legitimate, 2.9% fraud.

4.4 Data versus problem alignment

A key early finding was that the dataset does not contain the signals that define a SIM swap in production:

Available	Missing (SIM swap specific)
Amounts and sender/receiver balances	SIM swap event timestamp
Transaction type, device type, region	IMEI or device change flag
Hour, day of week	Account age or days since swap
Fraud label	Location shift, first transaction on a new device

Rather than abandon the problem, the project took what the notebook calls "Path B": engineer behavioural proxies from the balances, since a SIM swap drain leaves a distinctive footprint in how much money moves relative to what was there.

5. Methodology: The Five-Phase Build

The notebook is organised into five phases, each opening with a printed summary of tools, actions and results.

5.1 Phase 1: Data gathering, exploration and preparation

After loading and validating the data, exploration showed that the categorical features carry almost no signal: fraud rates sit at about 2.93% across every transaction type, device type and region, and fraud is spread evenly across the 24 hours. The signal lives in the balances.

Feature engineering: four SIM swap proxies

Feature	Definition	Correlation with fraud
balance_depletion_rate	amount / (sender_balance_before + 1)	0.50
sender_balance_ratio	sender_balance_after / (sender_balance_before + 1)	-0.62
is_balance_wipeout	1 if sender_balance_after < KES 100	0.78
is_high_value	1 if amount is in the top 10% (above KES 3,474)	0.09

The +1 in the denominators prevents division by zero for empty wallets. The raw balance features correlated with fraud at only about -0.19, so the engineered features were what made the problem learnable. Categorical columns were one-hot encoded (drop-first), and transaction_id and month_2026 were dropped, leaving 23 model features.

5.2 Phase 2: Visualisation and model selection

Phase 2 tested what kind of model the data needed:

- **No linear boundary.** Scatter plots showed fraud and legitimate transactions overlapping, with no straight line separating them.
- **Multicollinearity.** The two receiver balances correlate at 0.99, and wipeout and balance ratio at -0.74. Linear coefficients become unstable under this; tree splits do not.
- **Signal in combinations.** No single raw feature separated the classes, so models had to learn interactions.

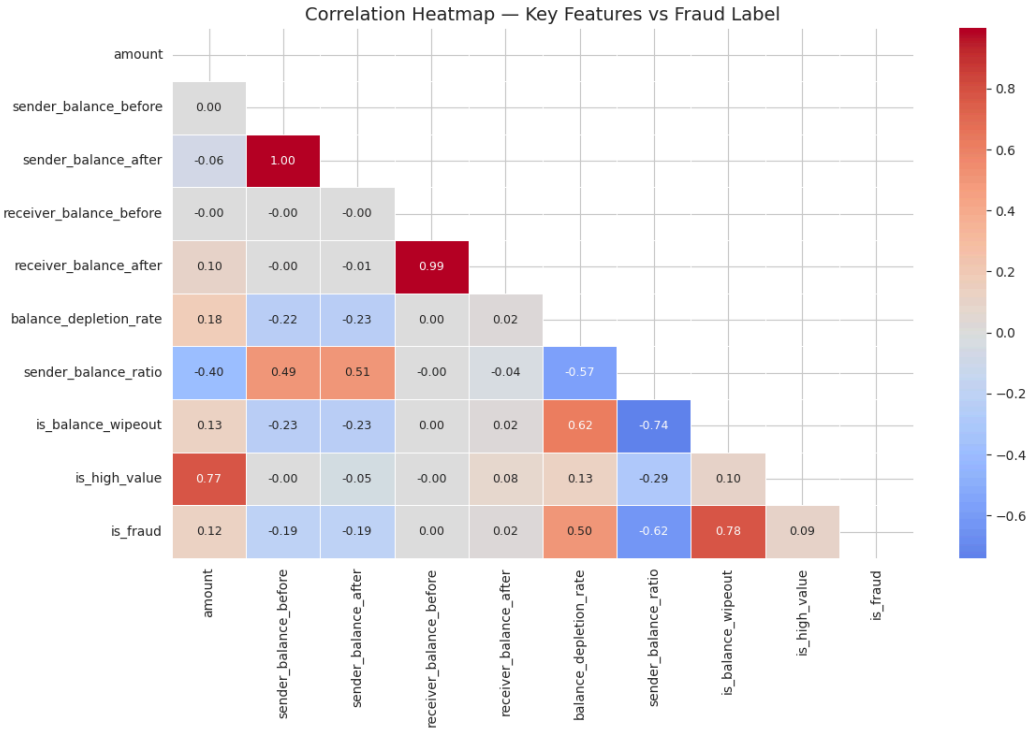


Figure 2. Correlation heatmap. The engineered wipeout and ratio features dominate the bottom row.

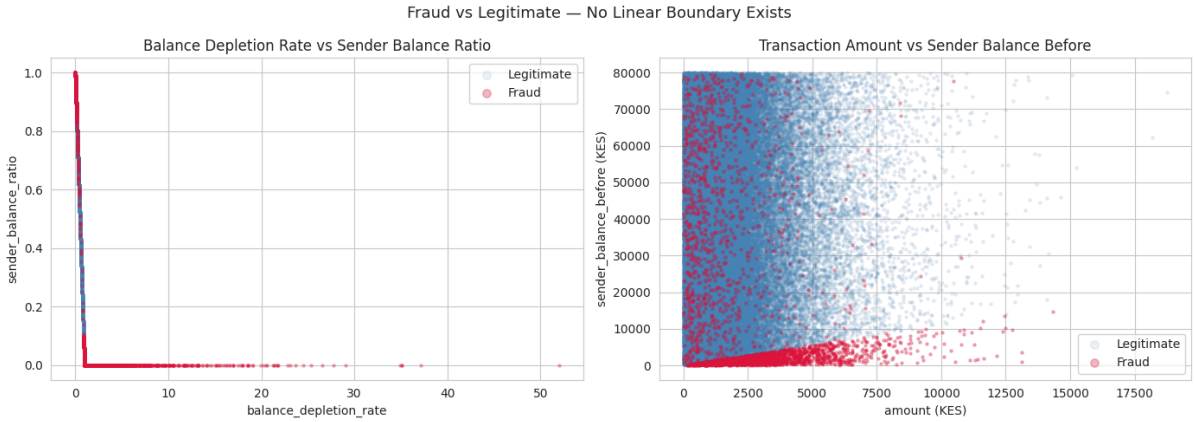


Figure 3. No linear boundary separates fraud (red) from legitimate (blue) transactions.

Decision: XGBoost as the primary model, LightGBM as a comparison, Logistic Regression as a deliberately simple baseline, and a WoE scorecard for interpretability in Phase 4.

5.3 Phase 3: Model training and testing

Split, scaling and imbalance handling

- Stratified 80/20 split: 95,999 training rows and 24,000 test rows, both with 2.93% fraud.
- StandardScaler fitted on training data only, then applied to the test set, to avoid leakage.
- SMOTE oversampling was tested and rejected. Class weighting (scale_pos_weight = 33 for the boosted trees, class_weight = balanced for logistic regression) gave clearly better precision and F1 with the same recall in that experiment.

Tuning

XGBoost and LightGBM were each tuned with RandomizedSearchCV: 30 parameter combinations, 5-fold cross-validation, scored on F1 (150 fits per model).

Model	Best parameters	CV F1
XGBoost	n_estimators 100, max_depth 4, learning_rate 0.05, subsample 1.0, colsample_bytree 1.0, min_child_weight 1	0.798
LightGBM	n_estimators 100, max_depth 5, num_leaves 15, learning_rate 0.01, subsample 0.8, colsample_bytree 0.6	0.785

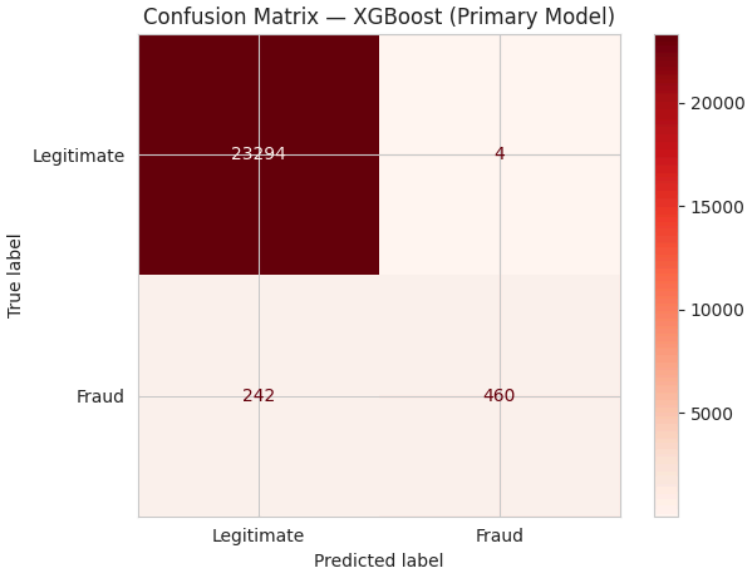


Figure 4. XGBoost confusion matrix: 460 fraud caught, 242 missed, 4 false positives.

Sanity check: the rule baseline

Because every model landed at almost the same recall (about 0.66) and AUC (about 0.83), a single-rule baseline was added: flag the transaction if the sender's balance after it is below KES 100.

Measure	Rule only	XGBoost
Precision	0.98	0.99
Recall	0.63	0.66
F1	0.76	0.79
Fraud caught	439	460
False positives	7	4

The rule and XGBoost agree on 99.87% of test rows. The fraud cases every model misses have ordinary amounts and healthy remaining balances, statistically indistinguishable from legitimate traffic. This points to how the synthetic data was generated more than to any model weakness, and it is the main reason recall could not reach the 0.80 target.

5.4 Phase 4: Regulatory interpretability layer

Gradient boosted trees give a probability but no plain-language reason. Kenyan lenders and regulators need to know why a transaction was flagged, so Phase 4 built a Weight of Evidence (WoE) logistic regression scorecard using scorecardpy.

Information Value (IV) feature selection

IV was computed on the training rows only.

Feature	IV	Reading
is_balance_wipeout	5.82	Extremely strong (99.2% of wiped-out transactions are fraud)
is_high_value	0.20	Medium
sender_balance_after	0.15	Medium
sender_balance_ratio	0.15	Medium
amount	0.098	Weak, kept as borderline (clean monotonic bins)
hour, sender_balance_before, balance_depletion_rate	< 0.01	Dropped as uninformative

Scorecard coefficients

A logistic regression trained on the five WoE features gives directly readable weights. The push on any one transaction is coefficient x WoE.

WoE feature	Coefficient	Interpretation
is_balance_wipeout	0.86	Dominant: a wipeout (WoE +8.33) adds about 7 to the log-odds
sender_balance_after	0.24	Balance below KES 4,000 (WoE +2.62) raises risk
sender_balance_ratio	0.13	Less than 74% of balance remaining (WoE +2.56)
is_high_value	0.13	Top 10% of amounts (WoE +1.01)
amount	0.11	Above KES 2,900 (WoE +0.92)

This lets a credit officer say, for example: "This transaction was flagged mainly because the sender’s account was wiped out, leaving under KES 100, and 99% of wiped-out transactions in the training data were fraud."

PSI drift monitoring

Population Stability Index was computed for each WoE feature, comparing training (baseline) against test (simulated production). All five features scored below 0.0001, well under the 0.10 monitoring threshold, establishing a clean baseline for future monthly checks.

5.5 Phase 5: Deployment

Architecture

The notebook exports everything the app needs into a deploy/ folder, copies it to Google Drive, and the folder is pushed to GitHub. Streamlit Community Cloud builds the app straight from the repository.

Path	Purpose
app.py	Streamlit app: feature engineering, scoring and all four pages
requirements.txt	Library versions pinned to the Colab training environment
.streamlit/config.toml	Forces the light theme so cards stay readable
models/xgboost_fraud_model.json	XGBoost model in version-safe JSON (pickle kept as fallback)
models/scaler.pkl, woe_scorecard_model.pkl	Fitted scaler and WoE logistic regression
models/woe_bins.json	WoE bin boundaries the app applies without scorecardpy
models/app_config.json	Thresholds, training ranges and test metrics read by the app

The app

The live app (open it here) has four pages, styled after the My Safaricom app:

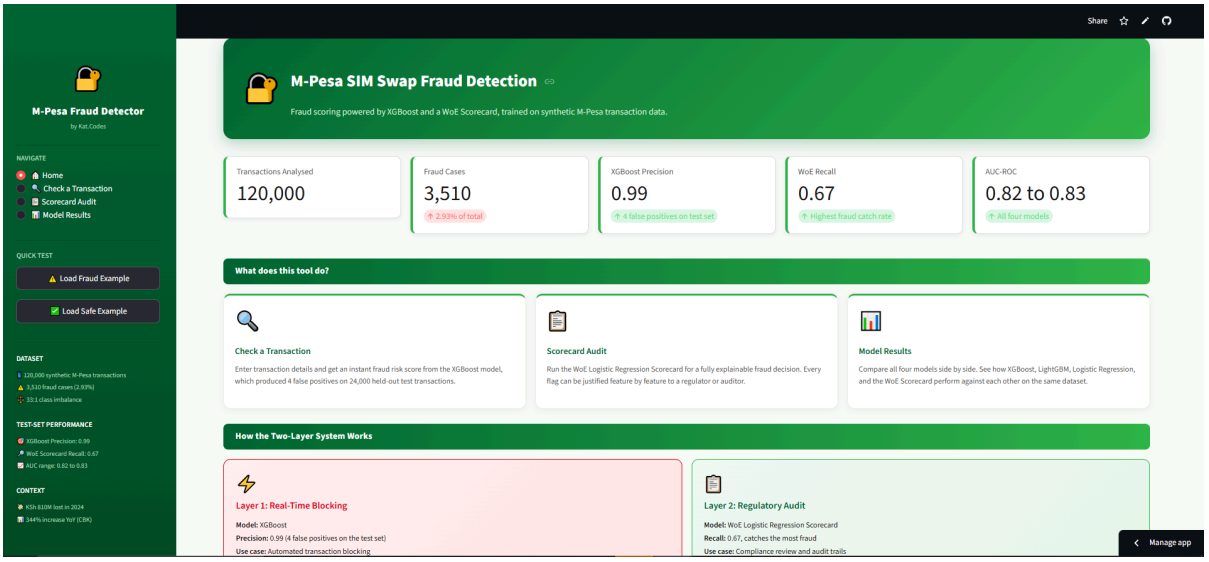


Figure 5. Home page: headline metrics and the two-layer system.

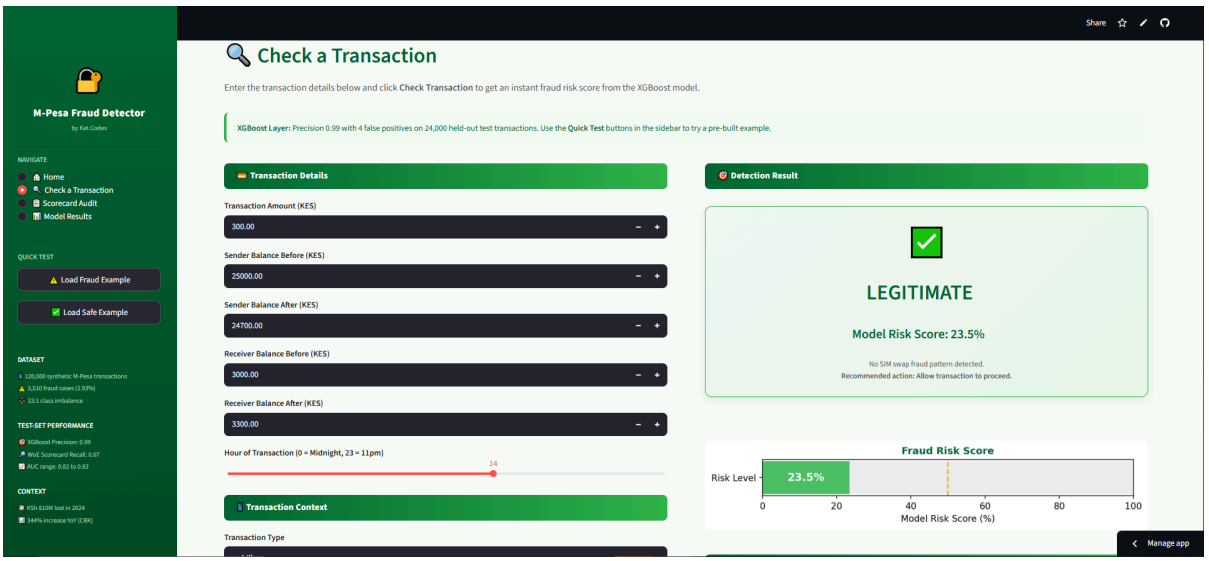


Figure 6. Check a Transaction: the XGBoost layer scores a legitimate example at 23.5% risk.

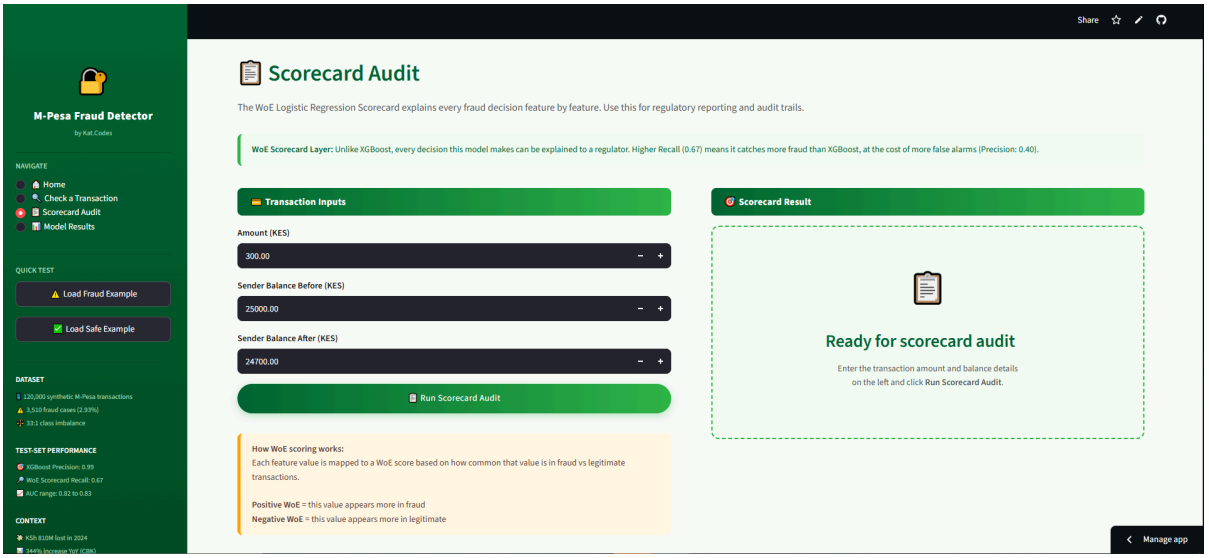


Figure 7. Scorecard Audit: the WoE layer explains each decision feature by feature.



Figure 8. Model Results: four-model comparison read from the saved test metrics.

6. Results

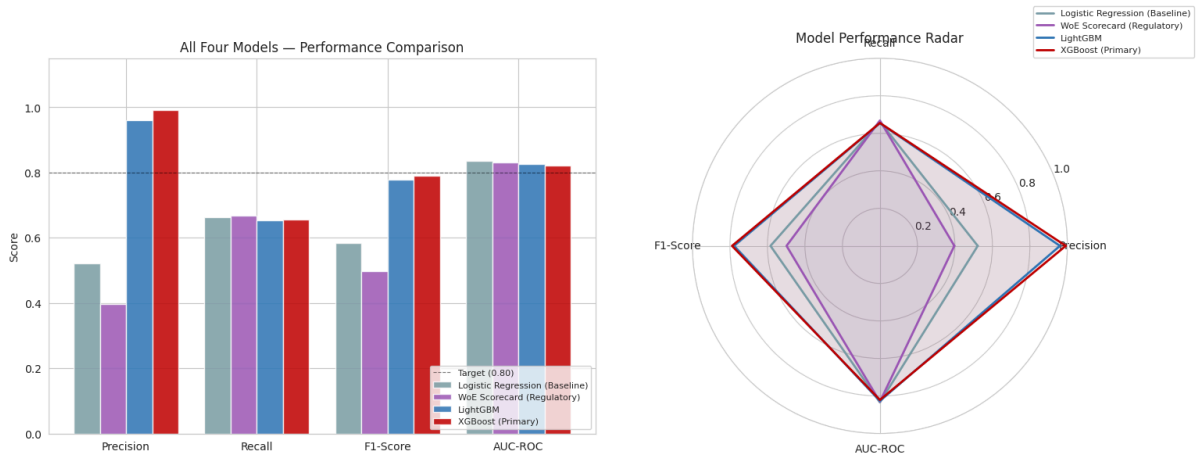


Figure 9. Four-model comparison across precision, recall, F1 and AUC-ROC.

Model	Precision	Recall	F1	AUC	FP	Fraud caught
Logistic Regression	0.52	0.66	0.58	0.83	426	465
WoE Scorecard	0.40	0.67	0.50	0.83	712	469
LightGBM	0.96	0.65	0.78	0.83	19	459
XGBoost	0.99	0.66	0.79	0.82	4	460

6.1 Interpretation

- **Same ranking ability, different risk tolerance.** All four models score AUC between 0.82 and 0.83. They differ mainly in where they set the decision boundary.
- **XGBoost for blocking.** Near-zero false positives make it suitable for automatic blocking, where wrongly stopping a customer is costly.
- **Scorecard for review and audit.** It catches the most fraud (469) and every flag can be explained, at the cost of 712 false alarms that suit a human review queue rather than automatic blocking.
- **Recommended architecture.** Run both together: XGBoost blocks high-confidence cases instantly, and the scorecard provides the wider net and the audit trail regulators expect.

7. Challenges and How They Were Resolved

The project ran from June to October 2026. The modelling came together early; most of the difficult problems surfaced during deployment and when the app was tested against the notebook. They are grouped below by stage.

7.1 Data and modelling

Challenge	What went wrong	Resolution
No SIM swap signals in the data	No timestamps, device or location fields to detect a swap directly	Engineered four balance-based behavioural proxies (Path B)
Severe class imbalance (33:1)	Accuracy-driven models would ignore fraud entirely	Stratified split, class weighting; SMOTE tested and rejected; accuracy excluded
Data leakage in the scorecard	WoE bins and IV were first fitted on all rows, including the test set	Split before binning; bins fitted on training rows only, with an assert that the split matches Phase 3
Strongest feature silently dropped	is_balance_wipeout had IV 5.8 but WoE 0. scorecardpy's 5% minimum bin size merged its 1.8% group away	Explicit break at 1. Scorecard precision rose from 0.23 to 0.40 and false positives fell from 1,629 to 712
Unreproducible "Precision 1.00"	A rerun gave 4 false positives instead of 0	Reported as 0.99 with a caveat; added the rule baseline to show what drives the result

7.2 Deployment and the Streamlit app

Challenge	What went wrong	Resolution
scorecardpy unavailable at runtime	It needs pkg_resources, which is missing in newer Python environments on Streamlit Cloud	WoE transformation re-implemented in the app from saved bin boundaries
Scorecard returned the same score for every input	The bin parser stripped "-inf" and "inf" to empty strings, so the lowest and highest bins never matched and returned WoE 0	Rewrote the parser using float("inf") and regex; verified the fraud example now gets WoE +2.62, +2.56, +8.33
Quick Test buttons did nothing	Widgets read their own session-state keys, so the buttons wrote values the widgets ignored	Buttons write directly to widget keys; values persist across page switches
Hard-coded threshold mismatch	App used KES 3,800 for "high value"; training actually used KES 3,474	Threshold, training ranges and metrics saved to app_config.json and read by the app

Challenge	What went wrong	Resolution
Invisible labels and cards	Forced white label text on a light page, and white metric text on white cards in dark mode	Scoped CSS to the sidebar, fixed metric colours, forced light theme via .streamlit/config.toml
Version mismatch risk	Unpinned requirements and a Python 3.11 setting against a Python 3.13 training environment	requirements.txt generated from the Colab versions; XGBoost and bins saved as JSON; Python 3.13 on Streamlit Cloud
Notebook failed on a fresh Run all	A cell loaded woe_bins.pkl before the cell that saved it	Check rewritten to use in-memory objects
Disorganised repository and slow deploys	All files at the top level; app stuck loading after the reorganisation	models/ folder with path-safe loading in app.py; redeploy pointing at the current repo and Python version

Lesson learned. The model was the easy part. Most of the risk sat in the handoff between notebook and app: the same feature engineering, thresholds, bin boundaries and library versions have to exist in both places. Saving these as configuration files, rather than retyping them, removed a whole class of bugs.

8. Limitations

- **Synthetic data.** The fraud labels appear to have been generated largely by draining balances, which makes the wipeout flag unrealistically clean. Real M-Pesa data will be noisier, and precision will drop.
- **No direct SIM swap signals.** The models detect the drain, not the swap. A real system would know a SIM was swapped minutes earlier.
- **Recall ceiling.** About one third of fraud is indistinguishable from legitimate traffic on the available features; tuning alone cannot fix this.
- **Uncalibrated scores.** Class weighting inflates the model output, so the app shows it as a "risk score" rather than a true probability.
- **Single transactions only.** There is no customer history, velocity (several transfers in minutes) or network view of receiving accounts.

9. Recommendations and Next Steps

1. **Add swap-aware features.** SIM swap recency (Safaricom already exposes a SIM-Swap-Check service to banks), device change flags, and time since last PIN reset.
2. **Add velocity and sequence features.** Number and value of transfers in the last 10 to 60 minutes, new recipients, and night-time bursts.
3. **Validate on real or semi-real data** before trusting the precision figures.
4. **Calibrate probabilities** (for example with CalibratedClassifierCV) and tune the decision threshold to business costs.
5. **Operational monitoring.** Weekly precision and recall checks, monthly PSI, and retraining when F1 falls below 0.70 or any PSI exceeds 0.25, as set out in the notebook's monitoring plan.

10. Tools and Environment

Area	Tools
Environment	Google Colab, Python 3.13, Google Drive
Data and analysis	pandas 2.2.3, NumPy 2.1.3, ydata-profiling
Visualisation	Matplotlib 3.10, Seaborn
Modelling	scikit-learn 1.6.1, XGBoost 3.4.1, LightGBM, scorecardpy 0.1.9.7
Deployment	Streamlit 1.66, GitHub, Streamlit Community Cloud, joblib

11. References and Links

Project

- Live app: https://sim-swap-fraud-detection-in-m-pesa-pzuecggagee65k9kdsyzs4.streamlit.app/
- GitHub repository: https://github.com/KathituCodes/SIM-Swap-Fraud-Detection-in-M-Pesa
- Dataset: https://www.kaggle.com/datasets/calebboen/mpesa-transactions-fraud

External validation

- Kenyans lose Sh1.59bn to bank fraud; mobile banking losses up 344% (TechTrends Kenya, Sept 2025)
- Hackers steal Sh1.59 billion from Kenya bank customers (Business Daily)
- INTERPOL report 2026: SIM swap fraud up 327% in Kenya (The Kenya Times, Aug 2026)
- How Kenyans lost Sh491m, cryptos via SIM hijack (Business Daily, Aug 2026)
- KSh 500m Fuliza fraud scheme unearthed, eight arrested (Citizen Digital, Feb 2023)
- Safaricom and DTB ordered to pay KSh 4.4m to SIM swap victim (HapaKenya, July 2026)
- Kenya, SA worst hit by mobile payment fraud: Evina data (Connecting Africa, May 2021)
- How M-Pesa SIM swap fraud works (Rest of World, March 2023)
- Mobile money is now Kenya's biggest fraud problem: NC4 report (Techweez, Aug 2026)